

Reviewer Pack — Minimal Rank of Hodge-Theoretic Motives over XOR-AND Tree Wi...

Entry 7d357d6dad22 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-27 11:22:22 UTC

Minimal Rank of Hodge-Theoretic Motives over XOR-AND Tree Width

Entry ID: 7d357d6dad22 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-27 11:22:22 UTC

1. Conjecture

Field A (mathematical branch): Arithmetic Geometry (Hodge-Theoretic Motives) **Field B** (complexity object): Complexity Theory: XOR-AND Tree Width

Statement:

[‘For every Boolean circuit C with n gates, the XOR-AND tree width of C is upper-bounded by $2^{\Omega(h(C))}$ where $h(C)$ is the arithmetic height of the Hodge-theoretic motive associated with C .’, ‘This implies that if two circuits have similar Hodge heights, their XOR-AND tree widths are comparable.’, ‘For any Boolean circuit C with n gates and a Hodge-height $h(C)$, there exists a finite field extension F such that the arithmetic height of the Hodge-theoretic motive over F is at most $2^{h(C)}$.’]

Rationale (proposer’s reasoning):

[‘Hodge-theoretic motives provide a rich algebraic structure that captures the complexity of geometric objects, which might expose hidden relationships between circuit complexity and algebraic invariants.’, ‘This conjecture connects arithmetic geometry with circuit complexity through XOR-AND tree width, which is a known lower-bound technique for monotone Boolean functions.’, ‘If true, it would offer a new perspective on understanding the hardness of certain types of Boolean circuits.’]

Taxonomy category: TSEITIN_RES (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 12db70e208ccab01

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, across 30 seeds, the mean XOR-AND tree width of circuits with Hodge height $h(C)$ is within a factor of $2^{(-\Omega(h(C)))}$ of the theoretical bound.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - arithmetic geometry AND hodge-theoretic motives AND XOR-AND tree width - Hodge-height AND Boolean circuit AND XOR-AND tree width - finite field extension AND Hodge-theoretic motive AND arithmetic height

Top relevant hits considered: - [http://arxiv.org/abs/0706.3841v1] Arithmetic lattices and weak spectral geometry - [http://arxiv.org/abs/2503.13307v2] On exact sequences of Hodge theoretic fundamental groups - [http://arxiv.org/abs/2603.05968v1] A Note on Hodge theoretic anabelian geometry - [http://arxiv.org/abs/1409.5893v2] Fast evaluation of far-field signals for time-domain wave propagation - [http://arxiv.org/abs/2604.22734v1] Radiation outer boundary conditions and near-to-far field signal transformations for the Bardeen-Press equation - [http://arxiv.org/abs/physics/0604089v3] Space-time transformation properties of inter-charge forces and dipole radiation: Breakdown of the classical field conce

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_circuit(n):
        if n == 1:
            return '0'
        else:
            left = generate_circuit(n // 2)
            right = generate_circuit(n - n // 2)
            return f'({left},{right})'

    def hodge_height(circuit):
        if circuit in ('0', '1'):
            return 0
        left, right = circuit.split(',')[0], circuit.split(',')[1][:-1]
        return max(hodge_height(left), hodge_height(right)) + 1
```

```

def xor_and_tree_width(circuit):
    if circuit in ('0', '1'):
        return 1
    left, right = circuit.split(',')[0], circuit.split(',')[1][:-1]
    return max(xor_and_tree_width(left), xor_and_tree_width(right))

n_values = [5, 10, 15, 20, 30, 40]
total_width = 0
instances_tested = 0

for n in n_values:
    for _ in range(5): # Ensure at least 30 instances per seed
        circuit = generate_circuit(n)
        height = hodge_height(circuit)
        width = xor_and_tree_width(circuit)
        total_width += width
        instances_tested += 1

mean_width = Fraction(total_width, instances_tested)
conjecture_holds = mean_width <= 2 ** (Fraction(1, 2) * math.log2(instances_tested))

return {
    "metric_name": "XOR-AND Tree Width",
    "metric_value": float(mean_width),
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_width = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_width} std=0.0 support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_width} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_84ef6ad1.py", line 69, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_84ef6ad1.py", line 48, in run_trial
    height = hodge_height(circuit)
             ^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_84ef6ad1.py", line 33, in hodge_height
    return max(hodge_height(left), hodge_height(right)) + 1
             ^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_84ef6ad1.py", line 32, in hodge_height
    left, right = circuit.split(',')[0], circuit.split(',')[1][:-1]
                                     ~~~~~^
IndexError: list index out of range
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete its execution to verify the conjecture. | next: Re-run the test with proper error handling and ensure that it completes without crashing.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11407
2	preregistration	ollama_remote	glm4:latest	0	0	5888
3	novelty	ollama_remote	glm4:latest	0	0	4679
4	novelty	ollama_remote	glm4:latest	0	0	6427
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12408
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8324
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8431
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8158
9	judge	ollama_remote	glm4:latest	0	0	7922

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 73644 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/7d357d6dad22.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/7d357d6dad22.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/7d357d6dad22.tar.gz` (if generated)